



Chapitre 3

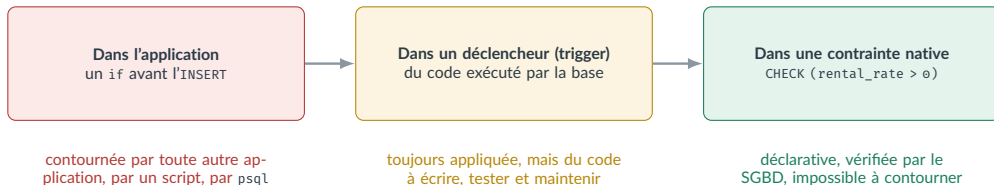
Contraintes et déclencheurs

Ce que la base *garantit*, quoi qu'il arrive



Trois façons d'exprimer des règles métier

« un tarif de location est strictement positif »



L'ordre de préférence à respecter :

1. contrainte native 2. déclencheur 3. application.

Quatre familles d'intégrité

Intégrité	Question	Outil
de domaine	cette valeur est-elle acceptable pour cette colonne ?	Type, NOT NULL, DEFAULT, CHECK
d'entité	cette ligne est-elle identifiable et unique ?	PRIMARY KEY, UNIQUE
référentielle	cette référence pointe-t-elle vers quelque chose qui existe ?	FOREIGN KEY
métier	cette règle propre à mon domaine est-elle respectée ?	CHECK <i>si la règle respecte certaines conditions</i> , sinon déclencheur

Remarque

Les trois premières familles sont couvertes par la modélisation. La quatrième nécessite des actions complémentaires, et c'est là que ce chapitre rentre en jeu.

Partie 1

Contraintes natives

ce que le SGBD sait garantir tout seul

PRIMARY KEY, UNIQUE, NOT NULL

garantir unicité et complétude

```
CREATE TABLE customer (  
  customer_id INTEGER PRIMARY KEY,  
  last_name VARCHAR(45) NOT NULL,  
  city VARCHAR(45),  
  active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

NOT NULL	la valeur doit être renseignée
DEFAULT	valeur prise si on ne dit rien
UNIQUE	pas deux fois la même valeur
PRIMARY KEY	UNIQUE + NOT NULL, une seule par table

UNIQUE et les valeurs absentes

Deux **NULL** ne sont pas « égaux ». Ils peuvent donc coexister dans une colonne UNIQUE.

email (UNIQUE)	accepté?
a@x.fr	✓
a@x.fr	✗ doublon
NULL	✓
NULL	✓ oui, encore

Si « chaque client a une adresse mail distincte » est la règle, il faut UNIQUE et NOT NULL.

UNIQUE (a, b) n'est pas UNIQUE (a) + UNIQUE (b)

attention au piège

Règle voulue : *un client ne donne qu'un seul avis par film.*

```
CONSTRAINT uq_review UNIQUE (film_id, customer_id)
```

film	client	accepté?
1	1	✓
1	2	✓ autre client
2	1	✓ autre film
1	1	✗ déjà existant

duplicate key value violates unique constraint "uq_review" - Key (film_id, customer_id)=(1, 1) already exists

À retenir

Une contrainte multi-colonnes porte sur le **couple**. Deux lignes ne sont en conflit que si elles coïncident sur *toutes* les colonnes citées.

Piège

UNIQUE (film_id) et UNIQUE (customer_id) séparément signifierait : « un seul avis par film au total » et « un seul avis par client au total ». Ce n'est pas du tout la même règle, et c'est une erreur de conception qu'on ne détecte parfois qu'en production.

CHECK : une condition sur la ligne

```
CREATE TABLE film (  
  length      INTEGER  
  CONSTRAINT ck_film_length CHECK (length > 0),  
  rating      VARCHAR(6)  
  CONSTRAINT ck_film_rating  
    CHECK (rating IN ('G', 'PG', 'PG-13', 'R', 'NC-17')),  
  rental_rate NUMERIC(4,2) NOT NULL  
  CONSTRAINT ck_film_rate CHECK (rental_rate > 0)  
);
```

Nommer ses contraintes n'est pas obligatoire, mais les messages d'erreur deviennent plus lisibles. Et on peut les supprimer par leur nom.

La règle d'évaluation

La ligne est rejetée seulement si la condition vaut **faux**.

Si elle vaut **inconnu** (parce qu'une valeur est **NULL**) la ligne passe.

length	length > 0	accepté ?
95	vrai	✓
0	faux	✗
-3	faux	✗
NULL	inconnu	✓

C'est l'inverse du **WHERE**, où « inconnu » écarte la ligne. Ici, « inconnu » la laisse entrer.

Ce qu'un CHECK ne peut pas faire

et c'est une des raisons d'être des déclencheurs

✗ Regarder une autre table

```
ALTER TABLE rental ADD CONSTRAINT ck_rental_client
CHECK (customer_id IN (SELECT customer_id FROM
customer));
```

cannot use subquery in check constraint

(et de toute façon, c'est le rôle d'une clé étrangère)

✗ Regarder les autres lignes

```
CHECK ((SELECT COUNT(*) FROM rental r
WHERE r.customer_id = customer_id
AND r.return_date IS NULL) <= 3)
```

cannot use subquery in check constraint

Le critère de décision de la séance

**La règle concerne-t-elle une seule ligne
d'une seule table ?**

Oui → contrainte native.

Non → déclencheur.

Exemples du TD :

tarif entre 0 et 9,99

date de retour $\geq$ date de location

deux catégories de même nom interdites

paiement $\leq 3 \times$ le tarif du film

au plus 3 locations en cours

un film déjà sorti n'est pas reloué

✓ native

✓ native

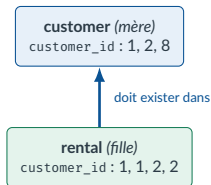
✓ native

✗ 2 tables

✗ n lignes

✗ n lignes

FOREIGN KEY : la référence doit exister



```
customer_id INTEGER NOT NULL  
CONSTRAINT fk_rental_customer  
REFERENCES customer(customer_id)  
ON DELETE CASCADE  
ON UPDATE CASCADE
```

La clé étrangère protège **deux** opérations :

- ▶ côté **filles** (table qui référence) : on ne peut pas insérer une location pour un client inexistant ;
- ▶ côté **mère** (table référencée) : on ne peut pas supprimer (ou renuméroter) un client qui a des locations... *sauf si on précise quoi faire.*

À retenir

C'est tout l'objet des clauses **ON DELETE** et **ON UPDATE**. Elles répondent à la question « **que deviennent les lignes filles si on modifie les lignes qu'elles référencent dans la table mère ?** »

Les modes d'intégrité référentielle

```
DELETE FROM customer WHERE customer_id = 2;
```

Mode	Effet sur <code>rental</code>	Quand le choisir
Défaut	✗ suppression refusée	un refus explicite plutôt qu'une perte de données
RESTRICT	✗ suppression refusée	idem, contrôle immédiat (non différable)
CASCADE	les locations du client 2 sont supprimées	la ligne fille n'a aucun sens sans sa mère
SET NULL	<code>customer_id</code> passe à NULL	la ligne fille garde un sens seule (trace)

Il existe aussi `SET DEFAULT`, rare : la valeur par défaut doit elle-même exister dans la table mère.

Piège

`SET NULL` sur une colonne `NOT NULL` : la contrainte est **acceptée à la création** et n'échoue qu'à la première suppression. *null value in column "customer_id" violates not-null constraint*

Le voir sur les données

deux modes, deux résultats : `DELETE FROM rental WHERE rental_id = 6;`

payment (avant)

id	rental	amount
5	6	3.99
13	17	4.99

La location 6 est référencée par le paiement 5.

ON DELETE CASCADE

id	rental	amount
5	6	3.99
13	17	4.99

Le paiement disparaît avec la location. La trace comptable est perdue.

ON DELETE SET NULL

id	rental	amount
5	NULL	3.99
13	17	4.99

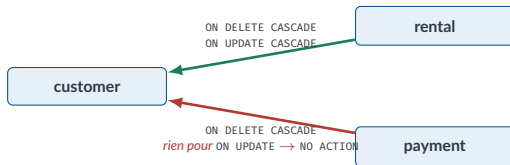
Le paiement survit, détaché : on sait qu'il a eu lieu, on ne sait plus à quelle location il se rapportait.

À retenir

Le choix n'est pas technique mais **métier** : « cette ligne a-t-elle encore un sens si sa mère disparaît ? » Répondre `CASCADE` par défaut, c'est accepter des suppressions en chaîne que personne n'a demandées.

Le mode se déclare clé par clé

et une seule suffit à tout bloquer



```
UPDATE customer SET customer_id = 99
WHERE customer_id = 7;
```

*update or delete on table "customer" violates foreign key constraint
"fk_pay_customer" on table "payment"*

La renumérotation échoue alors que la clé de **rental**,
elle, était bien en `ON UPDATE CASCADE`.

Piège

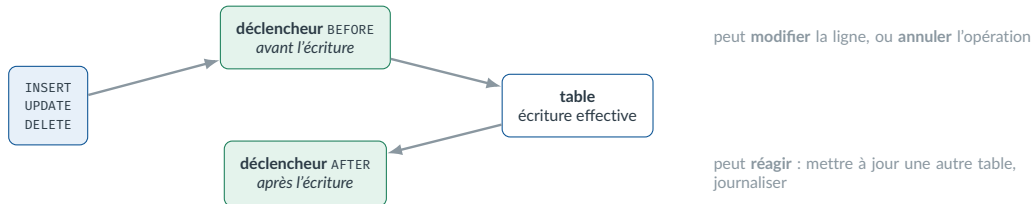
On regarde la clé étrangère qu'on a sous les yeux, on y lit `CASCADE`, on conclut. Il faut regarder **toutes** les tables qui référencent la table mère : le mode le plus restrictif l'emporte, et `ON DELETE` ne dit rien de `ON UPDATE`.

Partie 2

Déclencheurs (triggers)

du code qui s'exécute tout seul

Le principe



À retenir

Un déclencheur ne s'appelle pas : il se **dé-clenche** (comme son nom l'indique si bien). Quiconque écrit dans la table le subit : application, script de migration, ou vous en `psql`.

Remarque

C'est sa force et son danger : un traitement automatique et invisible. Un déclencheur non documenté est une source d'incompréhension durable pour l'équipe qui reprend la base.

Écrire un déclencheur : deux temps

1. Une fonction qui renvoie le type **TRIGGER**

```
CREATE OR REPLACE FUNCTION trg_film_dispo()
RETURNS TRIGGER AS $$
DECLARE v_nb INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_nb FROM rental
    WHERE film_id = NEW.film_id
        AND return_date IS NULL
        AND rental_id <> NEW.rental_id;
    IF v_nb > 0 THEN
        RAISE EXCEPTION 'Le film % est déjà loué', NEW.
            film_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

2. Le déclencheur qui l'attache

```
CREATE TRIGGER t_film_dispo
BEFORE INSERT OR UPDATE ON rental
FOR EACH ROW
EXECUTE FUNCTION trg_film_dispo();
```

BEFORE	→ moment d'exécution
INSERT OR UPDATE	→ cause de l'exécution
ON rental	→ table surveillée
FOR EACH ROW	→ une exécution par ligne impactée

La fonction n'a aucun paramètre : tout passe par les variables NEW, OLD et TG_OP.

NEW, OLD et TG_OP

trois variables, dont deux qui n'existent pas toujours

Opération	OLD	NEW	Valeur à renvoyer en BEFORE ... ROW
INSERT	NULL	la ligne à insérer	NEW (éventuellement modifiée)
UPDATE	la ligne avant	la ligne après	NEW (éventuellement modifiée)
DELETE	la ligne visée	NULL	OLD

À retenir

NEW et OLD sont des **lignes** : on lit `NEW.film_id`, et en BEFORE on peut écrire `NEW.amount := ...`

TG_OP contient 'INSERT', 'UPDATE' ou 'DELETE' en fonction de l'évènement qui a déclenché la fonction.

Piège

Utiliser NEW dans un déclencheur DELETE ne provoque **aucune erreur** : la variable vaut simplement **NULL**. Les conséquences arrivent à la diapo suivante.

La valeur de retour est la décision

un des points les plus importants du chapitre

	BEFORE ... FOR EACH ROW	AFTER ... FOR EACH ROW
RETURN NEW;	la ligne est écrite (avec vos modifications)	valeur ignorée (les modifications de NEW n'ont aucun effet)
RETURN NULL;	opération annulée pour cette ligne, <i>sans message</i>	valeur ignorée , la ligne reste écrite
RAISE EXCEPTION	tout est annulé, message d'erreur	tout est annulé, message d'erreur

Un filtre silencieux

```
IF NEW.amount = 0 THEN
  RETURN NULL;      -- la ligne ne sera pas insér
                    ée
END IF;
RETURN NEW;
```

À retenir

Le même code de fonction, attaché en BEFORE ou en AFTER, produit deux comportements **radicalement** différents.

Le mot-clé qui décide n'est pas dans la fonction : il est dans le CREATE TRIGGER.

BEFORE ou AFTER ? Un exemple qui tranche

on veut arrondir le montant avant enregistrement

```
CREATE FUNCTION trg_arrondi() RETURNS TRIGGER AS $$  
BEGIN  
    NEW.amount := ROUND(NEW.amount);  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

On insère un paiement de 4.99 :

déclencheur	montant enregistré
BEFORE INSERT	5.00
AFTER INSERT	4.99 (inchangé)

Avec AFTER, aucune erreur, aucun avertissement : la modification est simplement perdue.

La règle de choix

BEFORE : agir **sur** la ligne (compléter, normaliser, refuser).

AFTER : réagir à une écriture acquise (mettre à jour une autre table, journaliser).

Remarque

En AFTER, modifier la ligne exigerait un UPDATE explicite... qui redéclencherait le déclencheur, avec un risque de récursion. Raison de plus pour normaliser en BEFORE.

Trois usages typiques

1. Compléter une donnée

```
IF NEW.payment_date IS NULL
  THEN
    NEW.payment_date :=
      CURRENT_DATE;
END IF;
RETURN NEW;
```

BEFORE INSERT. Quand une simple clause DEFAULT ne suffit pas. Ici la valeur pourrait dépendre d'une autre table.

2. Interdire une écriture

```
IF v_nb >= 3 THEN
  RAISE EXCEPTION
    'Client % : 3 locations en
    cours',
    NEW.customer_id;
END IF;
RETURN NEW;
```

BEFORE INSERT OR UPDATE. C'est la « contrainte que CHECK ne sait pas exprimer ».

3. Maintenir une donnée dérivée

```
UPDATE customer
SET   nb_en_cours =
      nb_en_cours + 1
WHERE customer_id = NEW.
      customer_id;
RETURN NULL;
```

AFTER INSERT. On écrit dans une *autre* table : l'opération est acquise, le retour est ignoré.

Quatre erreurs qui ne provoquent aucun message

et comment s'en prémunir

Le code	Ce qu'on croit	Ce qui se passe
RETURN NEW; dans un BEFORE DELETE	la suppression a lieu	NEW vaut NULL ⇒ RETURN NULL ⇒ toutes les suppressions sont annulées , DELETE ☹
FOR EACH STATEMENT avec NEW	une exécution par ligne	une seule exécution par requête (et non par ligne), NEW vaut NULL
Déclencheur posé sur INSERT seulement	la règle est protégée	un UPDATE contourne la règle sans contrôle
COUNT(*) > 3 en BEFORE INSERT	refus à la 4 ^e ligne	la ligne courante n'est pas encore comptée : on en accepte une de trop

À retenir

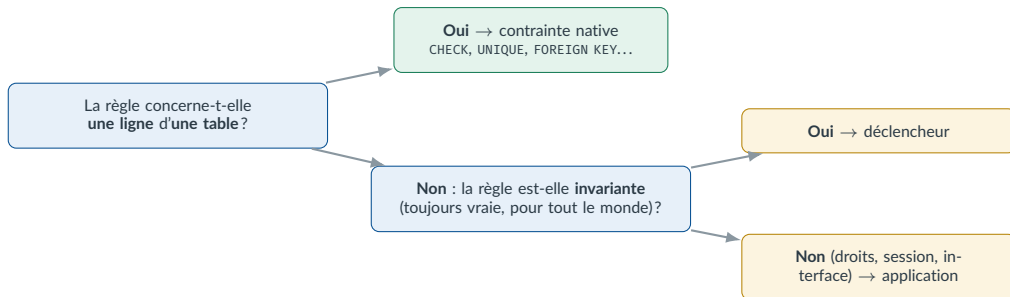
Trois de ces quatre erreurs viennent du même oubli : **le déclencheur s'exécute à un moment précis, sur une opération précise**. Avant d'écrire la logique, écrivez le CREATE TRIGGER et relisez-le.

Partie 3

Pour conclure

ça va, c'est fini maintenant !

Décider où mettre la règle



À retenir

Règle d'or : le déclaratif d'abord. Plus court, plus rapide, vérifié par le SGBD, et sans bug de logique possible.

Ce qu'il faut retenir

entre autres choses

Contraintes

1. Un CHECK ne rejette que ce qui est *faux* : **NULL** passe.
2. Un CHECK ne voit que la ligne courante : ni sous-requête, ni autre table.
3. Les modes ON DELETE/ON UPDATE se déclarent clé par clé ; regardez *toutes* les tables filles.

Déclencheurs

5. En BEFORE, la valeur renvoyée est la ligne : RETURN NULL annule en silence.
6. En AFTER, la valeur renvoyée est ignorée et NEW n'est plus modifiable.
7. NEW n'existe pas en DELETE, OLD pas en INSERT, et leur usage ne lève aucune erreur.
8. Une règle appliquée sur INSERT seulement se contourne par UPDATE.